## 一、安装 Qemu 工具

有两种方法可以在 Linux 环境下安装 Qemu 工具，第一种直接使用 XUbuntu 系统的 apt 工具安装，但是这种方法安装的 Qemu 系统版本不是最新的，如果需要安装最新版本的 Qemu 工具，就需要第二种方法，通过 Git 工具下载源码，切换到最新分支再去编译安装了；但是一般情况下通过 git 下载代码速度极慢，我们可以使用第三种方法，找到要下载的 qemu 版本，使用迅雷下载；具体操作如下所述：

### 1、快速安装 Qemu

**# sudo apt install qemu**

这种情况下安装的 qemu 版本可能不是最新版本；如果想要安装最新版本的 qemu，还得使用下边介绍的使用源代码编译安装的方法；

## 三、下载 Qemu 源码编译安装

### 1、下载 Qemu 源码

a、从 Git 服务器下载 Qemu 代码，记着在下载之前选择并切换需要的源码分支：

```
# git clone git://git.qemu-project.org/qemu.git # cd qemu # git checkout -b stable-***
remotes/origin/stable-***
```

b、在 windows 系统下使用迅雷下载

登陆 download.qemu.org 网站，选择需要的版本，点击下载，或者右键后选使用迅雷下载，速度会更快：

在这里选择 qemu-4.2.0.tar.xz 使用；

**2、安装**

在配置 qemu 之前，需要安装一些依赖的库或者软件包：

```
# sudo apt-get install zlib1g-dev # sudo apt-get install libglib2.0-0 # sudo apt-get install libglib2.0-dev # sudo apt-get install libtool # sudo apt-get install libsdl1.2-dev # sudo apt-get install autoconf
```

**解压源代码：**

```
# tar -xvf qemu-4.2.0.tar.xz
```

为了防止编译后文件比较乱，选择创建 build 目录作为编译中间目标路径：

```
# cd qemu-4.2.0/
# mkdir build
# cd build/
```

**配置、编译并安装 Qemu：**

```
# ../configure --target-list=arm-softmmu --audio-drv-list=
# make
# make install
```

### 3、在编译过程中可能出现的问题

```
# ../configure --target-list=arm-softmmu
--audio-drv-list=
ERROR: pkg-config binary 'pkg-config' not found
```

缺少库文件，按照上一步中的步骤安装库文件；

```
# ../configure --target-list=arm-softmmu
--audio-drv-list=
ERROR: pixman >= 0.21.8 not present.
Please install the pixman devel package.
```

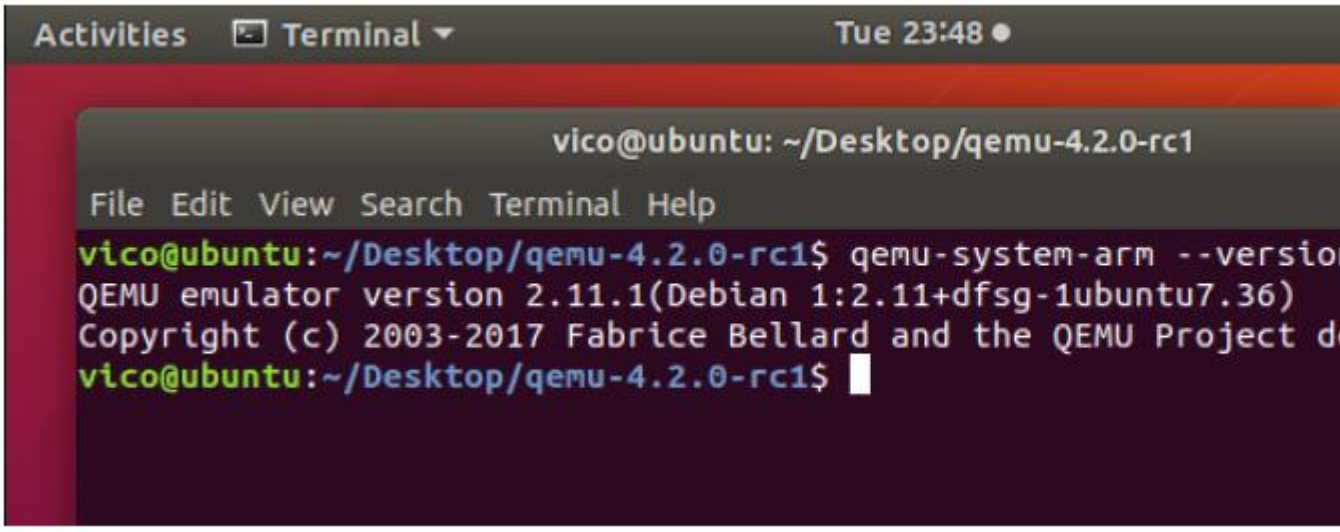
可以通过 apt-cache 查询缺少的依赖库：

```
# apt-cache search pixman
libpixman-1-0 - pixel-manipulation library for X and
cairo
libpixman-1-dev - pixel-manipulation library for X
and cairo (development files)
```

安装缺少的依赖库：

```
# sudo apt-get install libpixman-1-0
# sudo apt-get install libpixman-1-dev
```

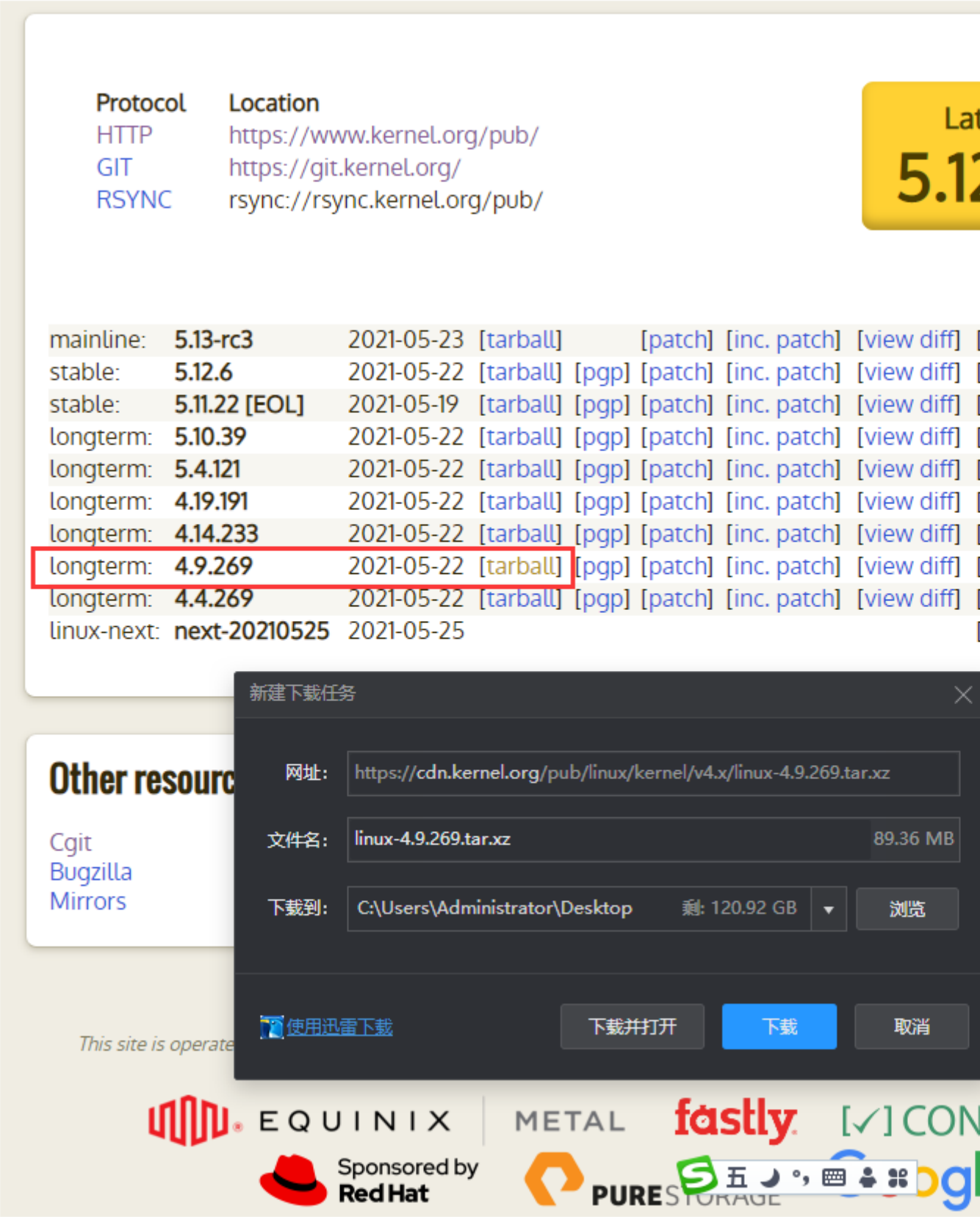
## 四、查看 Qemu 版本




## 五、下载源码且编译内核

1、下载地址：http://www.kernel.org

我们下载一个比较稳定的版本 4.9.269




因为我们此次使用模拟 arm 开发环境，所有我们需将内核编译成 arm 版本的内核

2、安装 arm-linuc-gcc　交叉编译工具，可以到网上去下载，版本如下

---

---

## 二、项目实战操作（mmap 映射物理内存到用户空间）

在内核申请一片物理内存，映射到用户空间使用的方法。

### 1、内核驱动模块 map.c

```
#include “.....”
#define MMAPIOMEM_DEV_NAME "mmapiomem"
```

```
#define MMAPIOMEM_DEV_MAJOR 280
#define MMAP_BUF_SIZE 0x500
char *mmap_buf_ptr;
int mmapiomem_open(struct inode *inode,struct file
*filp)
{
return 0;
}
int mmapiomem_release(struct inode *inode,struct
file *filp)
{
return 0;
}
int mmapiomem_mmap(struct file *filp,struct
vm_area_struct *vma)
{
int result;
unsigned long page;
vma->vm_flags |= (VM_IO | VM_LOCKED | (VM_DONTEXPAND
| VM_DONTDUMP));
vma->vm_flags|=VM_IO;
page = virt_to_phys(mmap_buf_ptr);
```

```
result = remap_pfn_range(vma, vma->vm_start,
((unsigned int)page)>>PAGE_SHIFT, PAGE_SIZE*4,
vma->vm_page_prot);
if(result)
{
return -EAGAIN;
}
return 0;
}
struct file_operations mmapiomem_fops={
.owner=THIS_MODULE,
.open=mmapiomem_open,
.release=mmapiomem_release,
.mmap=mmapiomem_mmap,
};
struct cdev *mmap_cdev;
struct class *mmap_class;
int mmapiomem_init(void)
{
int result;
int devno = MKDEV(MMAPIOMEM_DEV_MAJOR,0);
```

```
mmap_buf_ptr =
(char*)__get_free_pages(GFP_KERNEL,2);
printk("%s,the mmap_buf_ptr is
0x%p\n",__func__,mmap_buf_ptr);
memset(mmap_buf_ptr,0,PAGE_SIZE*4);
*((unsigned int *)(mmap_buf_ptr)) = 0xc3a21b44;
*((unsigned int *)(mmap_buf_ptr+0x4)) = 0xab8812df;
mmap_cdev = cdev_alloc();
result =
register_chrdev_region(devno,1,"mmap_char_mem");
cdev_init(mmap_cdev,&mmapiomem_fops);
mmap_cdev->owner = THIS_MODULE;
result = cdev_add(mmap_cdev, devno,1);
mmap_class =
class_create(THIS_MODULE,"mmap_char_class");
if (IS_ERR(mmap_class)) {
result= PTR_ERR(mmap_class);
return -1;
}
device_create(mmap_class, NULL, devno, NULL,
MMAPIOMEM_DEV_NAME);
return 0;
```

```
}
void mmapiomem_exit(void)
{
if (mmap_cdev != NULL)
cdev_del(mmap_cdev);
device_destroy(mmap_class,MKDEV(MMAPIOMEM_DEV_MAJO
R,0));
class_destroy(mmap_class);
unregister_chrdev_region(MKDEV(MMAPIOMEM_DEV_MAJOR
,0),1);
}
module_init(mmapiomem_init);
module_exit(mmapiomem_exit);
MODULE_LICENSE("GPL");
```

**2、Makefile：（文件设计）**

```
obj-m := map.o
all:
$(MAKE) -C /lib/modules/$(shell uname -r)/build
M=$(PWD) modules
clean:
$(MAKE) -C /lib/modules/$(shell uname -r)/build
M=$(PWD) clean
```

**3、简介如下**

主要是创建一个字符设备，配置其 mmap 函数，分配连续内存。分配了 4pages,并将首地址开始初始化了 8bytes。

具体使用方法是：make　　insmod map.ko　　dmesg-c

最后打印出：

```
Activities   Terminal ▾                                   Wed 01:27 ●
                                          vico@ubuntu: ~/Deskto
File  Edit  View  Search  Terminal  Help
[    14.020183] AVX2 version of gcm_enc/dec engaged
[    14.020184] AES CTR mode by8 optimization enable
[    37.828167] kauditd_printk_skb: 42 callbacks su
[    37.828169] audit: type=1400 audit(1622016254.6
profile_replace" profile="unconfined" name="snap-u
omm="apparmor_parser"
[    38.197088] audit: type=1400 audit(1622016255.0
profile_replace" profile="unconfined" name="snap.t
=1600 comm="apparmor_parser"
[    38.551064] audit: type=1400 audit(1622016255.3
profile_replace" profile="unconfined" name="snap.t
apparmor_parser"
[    38.877986] audit: type=1400 audit(1622016255.7
profile_replace" profile="unconfined" name="snap.t
comm="apparmor_parser"
[   146.874435] Bluetooth: RFCOMM TTY layer initial
[   146.874439] Bluetooth: RFCOMM socket layer init
[   146.874443] Bluetooth: RFCOMM ver 1.11
[   150.184338] rfkill: input handler disabled
[   989.900128] map: loading out-of-tree module tai
[   989.900153] map: module verification failed: si
 tainting kernel
[   989.900337] mmapiomem_init,the mmap_buf_ptr is
vico@ubuntu:~/Desktop/mtest$
```

4、lsmod 命令查看当前系统插入的模块信息

```
Activities   Terminal ▾                                   Wed 01:29 ●
                                          vico@ubuntu: ~/Deskto
File  Edit  View  Search  Terminal  Help
vico@ubuntu:~/Desktop/mtest$ ls
Makefile  map.ko   map.mod.c  map.o          Modul
map.c     map.mod  map.mod.o  modules.order
vico@ubuntu:~/Desktop/mtest$ lsmod
Module                  Size  Used by
map                    16384  0
rfcomm                 81920  4
intel_rapl_msr         20480  0
intel_rapl_common      24576  1 intel_rapl_msr
crct10dif_pclmul       16384  1
crc32_pclmul           16384  0
ghash_clmulni_intel    16384  0
aesni_intel           372736  0
vmw_balloon            24576  0
crypto_simd            16384  1 aesni_intel
cryptd                 24576  2 crypto_simd,ghash_
glue_helper            16384  1 aesni_intel
rapl                   20480  0
```

## 5、用户态测试操作技术

在虚拟机上测试结果如下：

```
root@ubuntu:~/2019/mmap/user_test# ./user
2) open the dev success
the value of ptrdata0 is 0x00
the value of ptrdata1 is 0x00
the value of ptrdata2 is 0x00
the value of ptrdata3 is 0x00
the value of ptrdata4 is 0x00
the value of ptrdata5 is 0x00
the value of ptrdata6 is 0x00
the value of ptrdata7 is 0x00
the addr map is: 0xf000000
```

可以看到在虚拟机下测试不成功，移步真机得到如下测试结果：

```
root@aaa:/home/aaa/2019/user_test# ./user
2) open the dev success
the value of ptrdata0 is 0x44
the value of ptrdata1 is 0x1b
the value of ptrdata2 is 0xa2
the value of ptrdata3 is 0xc3
the value of ptrdata4 is 0xdf
the value of ptrdata5 is 0x12
the value of ptrdata6 is 0x88
the value of ptrdata7 is 0xab
the addr_map is: 0xf000000
6)here close the dev
```

真机测试成功，可以看到，已经映射成功并且读取了我们之前初始化的值，返回了一个用户空间内存地址。因为程序结束后映射会取消掉，所以如果需要使用该地址的话，直接把这个addr_map 传给自己的其他接口函数就行。